

第十三周 概率算法 作业

1. 试设计一个算法随机地产生范围在1~n中的m个随机整数，且要求这m个随机整数互不相同。

```
void knuth(int n, int m)
{
    srand( (unsigned)time( NULL ) );
    for( int i = 0; i < n; i++)
    {
        if( rand()%(n-i) < m)
        {
            cout << i << "\t";
            m--;
        } // end if
    } //end for
    return;
}
```

虽然偶还是感觉都不应该调用rand，但是也行吧。。。

2. 试设计一个概率算法计算 $365! / 340! \times 365^{25}$ ，并精确到4位有效数字。

可参考 <https://zhuanlan.zhihu.com/p/513630744>

答：首先化简原计算公式：

$$\frac{365!}{340! \times 365^{25}} = \frac{341}{365} \times \frac{342}{365} \times \dots \times \frac{364}{365}$$

以24次随机试验为一次事件。第(365-K)次试验随机值取值范围是 [1, 365]，若实验值在 [1, K] 则为真，否则为假。若24次试验值都为真，则此次事件为真，否则为假，代码如下：

```
int solution(){
    static default_random_engine eng;
    static uniform_int_distribution<int> dis(1, 365);
    return dis(eng);
}

int main(){
    int t = 10;
    for(int i = 1; i <= t; ++i){
        int total = i * 1e7;
        int cnt = total;
        for(int j = 0; j < total; ++j){
            for(int k = 340; k <= 364; ++k){
                if(solution() > k){
                    cnt --;
                    break;
                }
            }
        }
        cout << "total: " << total << endl << "answer: " << double(cnt)/total;
    }
    return 0;
}
```

3. 设 p 是一个奇素数, $1 \leq x \leq p-1$, 如果存在一个整数 y , $1 \leq y \leq p-1$, 使得 $x = y^2 \pmod{p}$, 则称 y 是 x 的模 p 平方根。例如, 63是55的模103平方根。试设计一个求整数 x 的模 p 平方根的拉斯维加斯算法。

<https://baike.baidu.com/item/%E6%A8%A1%E5%B9%B3%E6%96%B9%E6%A0%B9/9126171>

托内利－尚克斯算法可以计算模平方根。算法的流程如下：

输入奇素数 p 和正整数 x ($1 \leq x \leq p-1$)

随机选取 g

设 $p-1 = 2g * t$, t 为奇数

$e=0$

for($i=2$; $i \leq s$; $i++$) if($(x * g^i - e)^{(p-1)/2^i} \neq 1$) $e += 2^i$

$h = x * g^{p-e}$

$b = (g^{e/2}) * h^{(t+1)/2}$

return b

托内利－尚克斯算法是概率算法，返回正确解的概率为 $1/2$ 。算法的渐进时间复杂度为 $O((\log p)^4)$ 。

4. 设计一个时间复杂度为 $O(n^2)$ 的算法来产生 $1, 2, \dots, n$ 的一个随机排列。要求用文字描述算法的基本思路，并分析其时间复杂度。

假设有 $1 \sim n$ 的一个序列。

每次选取 $1 \sim n$ 中一个 pos 作分割点

将 $1 \sim pos$ 与 $pos+1 \sim n$ 互换前后位置得到随机序列

n次找分割点, 复杂度 $O(N)$

```
Function (num[])  
    temp[n] // 数组  
    for (i=0; i<n; i++)  
        pos = random(1, n-1)  
        j = 1  
        for (k=pos+1; k<=n; k++, j++)  
            temp[j] = num[k]  
        for (k=1; k<=pos; k++, j++)  
            temp[j] = num[k]  
        for (k=1; k<=n; k++)  
            num[k] = temp[k]  
    return num; 返回随机序列
```

前后换位
复杂度 $O(N)$

总时间复杂度
 $O(n^2)$